



**UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
ESCOLA DO MAR, CIÊNCIA E TECNOLOGIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: HANDS-ON WORK VII**

EQUIPE

Felipe Ramos Silva – felipe.8517258@edu.univali.br

Felipe Ribeiro – felipe.7383142@edu.univali.br

Fernando Menezes de Jesus – fernando.8559341@edu.univali.br

João Gabriel Kuhn Burigo – joao.8518823@edu.univali.br

Letícia Lamara Vieira dos Anjos – leticia.8504059@edu.univali.br

Serviços Web — Transações Imobiliárias

ETAPA 02 – Partes 01 e 02 do Enunciado

Itajaí/SC, 2026

UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
ESCOLA DO MAR, CIÊNCIA E TECNOLOGIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: HANDS-ON WORK VII

EQUIPE

Felipe Ramos Silva – felipe.8517258@edu.univali.br

Felipe Ribeiro – felipe.7383142@edu.univali.br

Fernando Menezes de Jesus – fernando.8559341@edu.univali.br

João Gabriel Kuhn Burigo – joao.8518823@edu.univali.br

Letícia Lamara Vieira dos Anjos – leticia.8504059@edu.univali.br

Serviços Web — Transações Imobiliárias

ETAPA 02 – Partes 01 e 02 do Enunciado

Trabalho apresentado à disciplina Hands-On Work VII do curso de Análise e Desenvolvimento de Sistemas da Universidade do Vale do Itajaí – UNIVALI, sob orientação do Prof. Rafael Queiroz Gonçalves, como requisito parcial à obtenção da nota da Etapa 2.

Itajaí/SC, 2026

1. Modelagem do Banco de Dados (item a)

Estrutura das 3 tabelas: tipo imóvel, imóvel e pagamento com as respectivas chaves estrangeiras.

Definimos a ordem de criação das tabelas a partir das dependências de chave estrangeira: a tabela tipo_imovel não depende de nenhuma outra, a tabela imovel depende de tipo_imovel, e a tabela pagamento depende de imovel; por esse motivo, os comandos CREATE TABLE a seguir respeitam exatamente essa ordem. Optamos por utilizar o tipo DECIMAL(10,2), em vez de FLOAT, para o campo valor, uma vez que o tipo de ponto flutuante introduz erros de arredondamento inadequados à representação de valores monetários. Após a execução do script, verificamos manualmente, por meio do comando SHOW CREATE TABLE, se as chaves estrangeiras haviam sido criadas de fato, visto que um script pode ser executado sem erros mesmo quando uma chave estrangeira não é estabelecida corretamente.

```
CREATE DATABASE IF NOT EXISTS imobiliaria;
USE imobiliaria;

CREATE TABLE tipo_imovel (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(50) NOT NULL
);

CREATE TABLE imovel (
  id INT AUTO_INCREMENT PRIMARY KEY,
  descricao VARCHAR(200) NOT NULL,
  tipo_imovel_id INT NOT NULL,
  FOREIGN KEY (tipo_imovel_id) REFERENCES tipo_imovel(id)
);

CREATE TABLE pagamento (
  id INT AUTO_INCREMENT PRIMARY KEY,
  data_pagamento DATE NOT NULL,
  valor DECIMAL(10,2) NOT NULL,
  imovel_id INT NOT NULL,
  FOREIGN KEY (imovel_id) REFERENCES imovel(id)
);
```

2. Massa de Dados (itens b e c)

8 imóveis cadastrados, cada um com pelo menos 1 pagamento, e 32 pagamentos cobrindo 6 meses distintos.

Cadastramos 5 tipos de imóvel e 8 imóveis distribuídos entre esses tipos, além de 32 registros de pagamento distribuídos ao longo de 6 meses consecutivos, entre agosto de 2023 e janeiro de 2024, quantidades superiores aos mínimos de 30 pagamentos e 5 meses distintos exigidos no enunciado. Tivemos o cuidado de garantir que todos os 8 imóveis possuísem ao menos um pagamento associado, conforme exigido no item (c).

```

USE imobiliaria;

-- 1. Tipos de imovel
INSERT INTO tipo_imovel (nome) VALUES
('Apartamento'),
('Casa'),
('Sala Comercial'),
('Terreno'),
('Galpao');

-- 2. Imoveis (8 no minimo, tipos variados)
INSERT INTO imovel (descricao, tipo_imovel_id) VALUES
('Apartamento 100m2 em condominio fechado', 1),
('Apartamento 60m2 no centro', 1),
('Casa 3 quartos com quintal', 2),
('Casa geminada 2 quartos', 2),
('Sala comercial 40m2 no centro', 3),
('Sala comercial em galeria', 3),
('Terreno 500m2 em bairro residencial', 4),
('Galpao industrial 800m2', 5);

-- 3. Pagamentos (32 registros, 6 meses distintos)
INSERT INTO pagamento (data_pagamento, valor, imovel_id) VALUES
('2023-08-10', 5000.00, 1), ('2023-08-15', 4800.00, 2),
('2023-08-20', 3200.00, 3), ('2023-08-25', 3100.00, 4),
('2023-09-05', 2500.00, 5), ('2023-09-10', 2600.00, 6),
('2023-09-15', 7000.00, 7), ('2023-09-20', 9500.00, 8),
('2023-09-25', 5100.00, 1), ('2023-10-01', 4900.00, 2),
('2023-10-05', 3300.00, 3), ('2023-10-10', 3150.00, 4),
('2023-10-15', 2550.00, 5), ('2023-10-20', 2650.00, 6),
('2023-10-25', 7100.00, 7), ('2023-11-01', 9600.00, 8),
('2023-11-05', 5200.00, 1), ('2023-11-10', 5000.00, 2),
('2023-11-15', 3400.00, 3), ('2023-11-20', 3200.00, 4),
('2023-11-25', 2600.00, 5), ('2023-12-01', 2700.00, 6),
('2023-12-05', 7200.00, 7), ('2023-12-10', 9700.00, 8),
('2023-12-15', 5300.00, 1), ('2023-12-20', 5100.00, 2),
('2023-12-25', 3500.00, 3), ('2024-01-05', 3300.00, 4),
('2024-01-10', 2700.00, 5), ('2024-01-15', 2800.00, 6),
('2024-01-20', 7300.00, 7), ('2024-01-25', 9800.00, 8);

```

3. Consulta SQL com JOIN (item d)

Consulta única com INNER JOIN entre as 3 tabelas, sem WHERE nem GROUP BY, conforme exigido no enunciado.

Utilizamos INNER JOIN, e não LEFT JOIN, pois o modelo de dados garante que todo pagamento está associado a um imóvel e todo imóvel a um tipo, não havendo necessidade de tratar valores nulos nessas relações. Definimos os nomes das colunas de saída (id_venda, data_do_pagamento, valor_do_pagamento, codigo_imovel, descricao_imovel, tipo_imovel) de modo a espelhar a estrutura de exemplo apresentada no enunciado.

```
SELECT
    pagamento.id AS id_venda,
    pagamento.data_pagamento AS data_do_pagamento,
    pagamento.valor AS valor_do_pagamento,
    imovel.id AS codigo_imovel,
    imovel.descricao AS descricao_imovel,
    tipo_imovel.nome AS tipo_imovel
FROM pagamento
INNER JOIN imovel
    ON pagamento.imovel_id = imovel.id
INNER JOIN tipo_imovel
    ON imovel.tipo_imovel_id = tipo_imovel.id;
```

4. Código-fonte — Consulta via Node.js (item e)

Código que executa a consulta do item d a partir da aplicação.

O script conecta-se ao banco de dados por meio de um pool de conexões (mysql2/promise), realiza a leitura do arquivo db/query_join.sql e executa a consulta especificada no item (d), exibindo o total de linhas retornadas e uma amostra do resultado. Durante o desenvolvimento, identificamos que o driver mysql2 retorna colunas do tipo DECIMAL como texto (por exemplo, "5000.00" em vez do valor numérico 5000); caso não tratada, essa característica provocaria concatenação de strings em vez de soma nas funções de agregação previstas para a Etapa 2. Resolvemos essa questão na camada de modelos do projeto, documentada em src/models/README.md, de modo que a conversão seja reaproveitada pelos três serviços da Etapa 2.

```
const fs = require('fs');
const path = require('path');

// Carrega o .env da raiz do projeto, não da pasta teste/
require('dotenv').config({ path: path.join(__dirname, '..', '.env') });

// Sobe um nível (..) para achar src/db/connection.js
const pool = require('../src/db/connection');
```

```

async function testar() {
  try {
    // Sobe um nível (..) para achar db/query_join.sql
    const caminhoQuery = path.join(__dirname, '..', 'db', 'query_join.sql');
    const query = fs.readFileSync(caminhoQuery, 'utf8');

    const [linhas] = await pool.query(query);

    console.log('Conexão com o MySQL: OK');
    console.log('Total de linhas:', linhas.length);
    console.table(linhas.slice(0, 5));
  } catch (erro) {
    console.error('Deu erro:', erro.message);
  } finally {
    await pool.end();
  }
}

testar();

```

5. Especificação OpenAPI (item f)

Descrição dos 3 serviços REST que serão implementados na Etapa 2 (Parte 2).

Especificamos, no documento a seguir, o contrato dos três serviços REST que serão implementados na Etapa 2, soma por imóvel, total de vendas por mês e percentual de vendas por tipo de imóvel, previamente à implementação do código correspondente. Essa definição antecipada permite que os integrantes responsáveis por cada endpoint na Etapa 2 desenvolvam seus respectivos serviços de forma independente, conhecendo previamente o formato de resposta esperado para cada rota.

```
openapi: 3.0.3
```

```
info:
```

```
  title: API de Transacoes Imobiliarias
```

```
  description: Servicos REST para consulta de transacoes imobiliarias.
```

```
  version: 1.0.0
```

```
servers:
  - url: http://localhost:3000

paths:

  /api/soma-por-imovel:
    get:
      summary: Soma das vendas por imovel
      responses:
        '200':
          description: Lista com o total de vendas por imovel.
          content:
            application/json:
              example:
                - codigo_imovel: 1
                  descricao_imovel: "Apartamento 100m2 em condominio fechado"
                  total_vendas: 20400.00

  /api/vendas-por-mes:
    get:
      summary: Total de vendas por mes e ano
      responses:
        '200':
          description: Lista com o total de vendas agrupadas por mes e ano.
          content:
            application/json:
              example:
                - ano: 2023
                  mes: 8
                  total_vendas: 16100.00

  /api/vendas-por-tipo:
    get:
      summary: Percentual de vendas por tipo de imovel
      responses:
        '200':
          description: Lista com o percentual de vendas por tipo de imovel.
          content:
            application/json:
              example:
                - tipo_imovel: "Apartamento"
                  percentual: 25.0
```

6. Arquitetura da Aplicação — Orientação a Objetos e Programação Funcional

Organização em camadas: rota → controller → model → conexão com o banco de dados.

Conforme exigido no enunciado, os serviços empregam os dois paradigmas de programação. O paradigma orientado a objetos está presente nas classes de domínio (TipoImovel, Imovel e Pagamento), que representam as entidades das tabelas, e nas classes de controle (ImovelController, VendaMensalController e TipoImovelController), que concentram a lógica de cada serviço. O paradigma funcional está no processamento: os dados são carregados na íntegra pelo SQL, sem cláusulas WHERE ou GROUP BY, e todos os filtros e agrupamentos são feitos na linguagem de programação com map e reduce.

Cada requisição percorre as camadas na mesma ordem. A rota Express apenas recebe a chamada GET e devolve o JSON; o controller executa a consulta, converte cada linha em um objeto de model por meio do método estático `fromDatabase` e realiza o cálculo; o model garante a tipagem dos campos, convertendo em número (`Number()`) os valores do tipo DECIMAL, que o driver mysql2 entrega como texto; e a conexão fornece o pool de acesso ao MySQL.

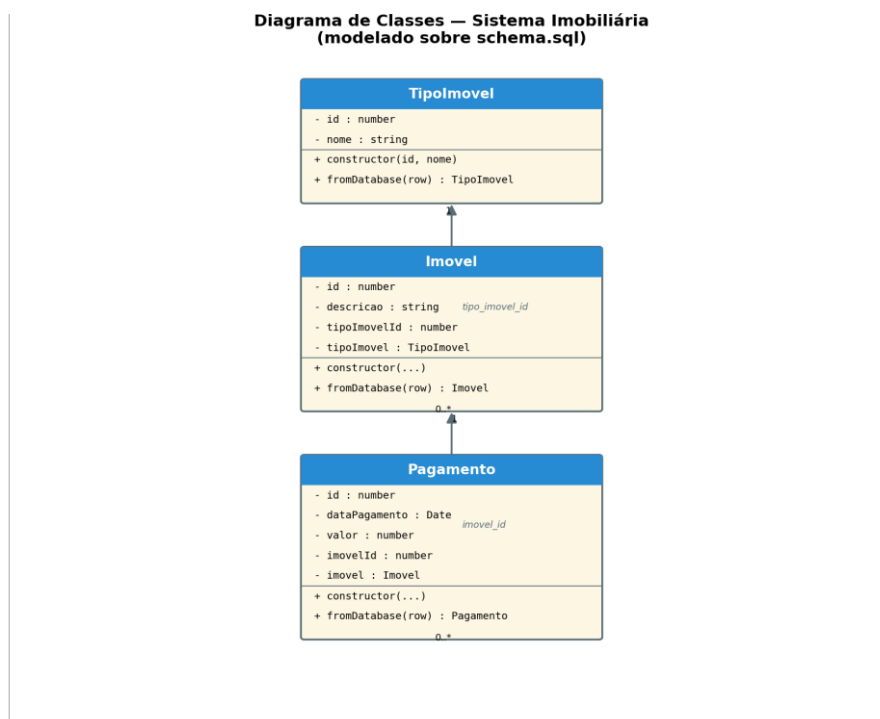


Figura 1 — Diagrama de classes UML das entidades do domínio

src/models/TipoImovel.js

```
class TipoImovel {
  constructor(id, nome) {
    this.id = id;
    this.nome = nome;
  }

  static fromDatabase(row) {
    return new TipoImovel(
      Number(row.id),
      row.nome
    );
  }
}
```

```
module.exports = TipoImovel;
```

src/models/Imovel.js

```
const TipoImovel = require('./TipoImovel');
```

```
class Imovel {
  constructor(id, descricao, tipoImovelId, tipoImovel = null) {
    this.id = id;
    this.descricao = descricao;
    this.tipoImovelId = tipoImovelId;
    this.tipoImovel = tipoImovel;
  }

  static fromDatabase(row) {
    if (row.codigo_imovel !== undefined) {
      const tipo = row.tipo_imovel !== undefined
        ? new TipoImovel(null, row.tipo_imovel)
        : null;
      return new Imovel(
        Number(row.codigo_imovel),
        row.descricao_imovel,
        tipo ? tipo.id : null,
        tipo
      );
    }

    return new Imovel(
      Number(row.id),
      row.descricao,
      row.tipo_imovel_id !== undefined ? Number(row.tipo_imovel_id) : null
    );
  }
}
```

```
module.exports = Imovel;
```

src/models/Pagamento.js

```
const Imovel = require('./Imovel');
```

```
class Pagamento {
  constructor(id, dataPagamento, valor, imovelId, imovel = null) {
    this.id = id;
    this.dataPagamento = dataPagamento;
    this.valor = Number(valor);
    this.imovelId = imovelId;
    this.imovel = imovel;
  }
}
```

```

    static fromDatabase(row) {
      if (row.id_venda !== undefined) {
        const imovel = Imovel.fromDatabase(row);
        return new Pagamento(
          Number(row.id_venda),
          row.data_do_pagamento,
          Number(row.valor_do_pagamento),
          imovel.id,
          imovel
        );
      }

      return new Pagamento(
        Number(row.id),
        row.data_pagamento,
        Number(row.valor),
        row.imovel_id !== undefined ? Number(row.imovel_id) : null
      );
    }
  }
}

module.exports = Pagamento;

```

src/db/connection.js — pool de conexão com o MySQL

```

const path = require('path');
require('dotenv').config({ path: path.join(__dirname, '..', '..', '.env') });
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  charset: 'utf8mb4',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;

```

src/server.js — servidor Express e registro das três rotas

```

require('dotenv').config();
const express = require('express');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.get('/', (req, res) => {
  res.json({ status: 'ok', servico: 'how7-backend-imobiliario' });
});

const vendasPorImovel = require('./routes/vendasPorImovel');
app.use('/api/soma-por-imovel', vendasPorImovel);
const vendasPorMes = require('./routes/vendasPorMes');
app.use('/api/vendas-por-mes', vendasPorMes);
const vendasPorTipo = require('./routes/vendasPorTipo');
app.use('/api/vendas-por-tipo', vendasPorTipo);

```

```
app.listen(PORT, () => {  
  console.log(`Servidor rodando em http://localhost:${PORT}`);  
});
```

7. Serviço A — Soma dos Pagamentos por Imóvel (item a)

GET /api/soma-por-imovel — dados para o gráfico de barras.

A função executa a consulta com JOIN do item (d), converte cada linha em um objeto Pagamento com map e, em seguida, acumula com reduce o valor de todos os pagamentos de cada imóvel, considerando toda a série histórica. Um map final arredonda o total para duas casas decimais. O retorno é uma lista com o código, a descrição e o total de vendas de cada um dos 8 imóveis.

src/controllers/ImovelController.js

```
const fs = require('fs');
const path = require('path');
const pool = require('../db/connection');
const Pagamento = require('../models/Pagamento');

const QUERY_JOIN = fs.readFileSync(
  path.join(__dirname, '..', '..', 'db', 'query_join.sql'),
  'utf8'
);

class ImovelController {
  static async somaPorImovel() {
    const [rows] = await pool.query(QUERY_JOIN);
    const pagamentos = rows.map((row) => Pagamento.fromDatabase(row));

    const totaisPorImovel = pagamentos.reduce((acumulado, pagamento) => {
      const { id, descricao } = pagamento.imovel;

      if (!acumulado[id]) {
        acumulado[id] = {
          codigo_imovel: id,
          descricao_imovel: descricao,
          total_vendas: 0,
        };
      }

      acumulado[id].total_vendas += pagamento.valor;
      return acumulado;
    }, {});

    return Object.values(totaisPorImovel).map((item) => ({
      ...item,
      total_vendas: Math.round(item.total_vendas * 100) / 100,
    }));
  }
}

module.exports = ImovelController;
```

src/routes/vendasPorImovel.js

```
const express = require('express');
const ImovelController = require('../controllers/ImovelController');

const router = express.Router();

router.get('/', async (req, res) => {
  try {
```

```

    const resultado = await ImovelController.somaPorImovel();
    res.status(200).json(resultado);
  } catch (erro) {
    res.status(500).json({ erro: erro.message });
  }
});

```

```
module.exports = router;
```

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3000/api/soma-por-imovel
- Status:** 200 OK
- Response Time:** 3 ms
- Response Size:** 972 B
- Response Format:** JSON

The response body contains an array of 8 objects, each representing a property with its code, description, and total sales value.

codigo_imovel	descricao_imovel	total_vendas
1	Apartamento 100m2 em condominio fechado	20600
2	Apartamento 60m2 no centro	19800
3	Casa 3 quartos com quintal	13400
4	Casa geminada 2 quartos	12750
5	Sala comercial 40m2 no centro	10350
6	Sala comercial em galeria	10750
7	Terreno 500m2 em bairro residencial	28600
8	Galpão industrial 800m2	38600

Figura 2 — Teste do serviço A: GET /api/soma-por-imovel (200 OK)

8. Serviço B — Total de Vendas por Mês/Ano (item b)

GET /api/vendas-por-mes — dados para o gráfico de linhas.

A função carrega todos os pagamentos sem filtro nem agrupamento no SQL, converte cada linha em Pagamento com map e usa reduce para somar os valores de cada período no formato MM/AAAA. O mês e o ano são obtidos em UTC (getUTCMonth e getUTCFullYear), para que o fuso horário do servidor não desloque um pagamento do dia 1º para o mês anterior. O retorno lista os 6 meses com vendas registradas e o respectivo total.

src/controllers/VendaMensualController.js

```
const pool = require('../db/connection');
const Pagamento = require('../models/Pagamento');

class VendaMensualController {
  static async vendasPorMes() {
    const [rows] = await pool.query(`
      SELECT id, data_pagamento, valor, imovel_id
      FROM pagamento
    `);

    const pagamentos = rows.map(row => Pagamento.fromDatabase(row));

    const vendasPorMes = pagamentos.reduce((acumulador, pagamento) => {
      const data = new Date(pagamento.dataPagamento);

      const mes = String(data.getUTCMonth() + 1).padStart(2, '0');
      const ano = data.getUTCFullYear();

      const periodo = `${mes}/${ano}`;

      if (!acumulador[periodo]) {
        acumulador[periodo] = 0;
      }

      acumulador[periodo] += pagamento.valor;

      return acumulador;
    }, {});

    return Object.entries(vendasPorMes).map(([mesAno, total]) => ({
      mesAno,
      total: Number(total.toFixed(2))
    }));
  }
}

module.exports = VendaMensualController;
```

src/routes/vendasPorMes.js

```
const express = require('express');

const VendaMensualController = require('../controllers/VendaMensualController');

const router = express.Router();

router.get('/', async (req, res) => {
```

```

try {
  const resultado = await VendaMensalController.vendasPorMes();
  res.status(200).json(resultado);
} catch (erro) {
  res.status(500).json({ erro: erro.message });
}
});

module.exports = router;

```

New Collection > **New Request** Save Share

GET http://localhost:3000/api/vendas-por-mes Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK 3 ms 449 B Save Response

JSON Preview Visualize

```

1  [
2    {
3      "mesAno": "08/2023",
4      "total": 16100
5    },
6    {
7      "mesAno": "09/2023",
8      "total": 26700
9    },
10   {
11     "mesAno": "10/2023",
12     "total": 23650
13   },
14   {
15     "mesAno": "11/2023",
16     "total": 29000
17   },
18   {
19     "mesAno": "12/2023",
20     "total": 33500
21   },
22   {
23     "mesAno": "01/2024",
24     "total": 25900
25   }
26 ]

```

Figura 3 — Teste do serviço B: GET /api/vendas-por-mes (200 OK)

9. Serviço C — Percentual de Vendas por Tipo de Imóvel (item c)

GET /api/vendas-por-tipo — dados para o gráfico de pizza.

A função executa a consulta com JOIN do item (d), converte as linhas em Pagamento e usa reduce para acumular, por tipo de imóvel, o valor total e a quantidade de vendas. Dois novos reduce calculam o valor geral e a quantidade geral, e um map final produz, para cada tipo, o percentual sobre o total, arredondado para duas casas decimais.

Decisão sobre a interpretação do item (c). O item (c) da Parte 2 pede o percentual “no total de vendas (quantitativas)”, enquanto a descrição do gráfico de pizza fala em “percentual do valor total das vendas”. Como as duas leituras produzem resultados diferentes, o serviço devolve ambas: percentualPorValor, calculado sobre a soma dos valores, e percentualPorQuantidade, calculado sobre o número de vendas. Assim, o frontend pode montar o gráfico com a leitura que o cliente escolher.

`src/controllers/TipoImovelController.js`

```
const fs = require('fs');
const path = require('path');
const pool = require('../db/connection');
const Pagamento = require('../models/Pagamento');

const QUERY_JOIN = fs.readFileSync(
  path.join(__dirname, '..', '..', 'db', 'query_join.sql'),
  'utf8'
);

class TipoImovelController {
  static async percentualPorTipo() {
    const [rows] = await pool.query(QUERY_JOIN);
    const pagamentos = rows.map((row) => Pagamento.fromDatabase(row));

    const acumuladoPorTipo = pagamentos.reduce((acumulado, pagamento) => {
      const tipo = pagamento.imovel.tipoImovel.nome;

      if (!acumulado[tipo]) {
        acumulado[tipo] = { totalValor: 0, quantidadeVendas: 0 };
      }

      acumulado[tipo].totalValor += pagamento.valor;
      acumulado[tipo].quantidadeVendas += 1;

      return acumulado;
    }, {});

    const valorGeral = Object.values(acumuladoPorTipo)
      .reduce((soma, item) => soma + item.totalValor, 0);
    const quantidadeGeral = Object.values(acumuladoPorTipo)
      .reduce((soma, item) => soma + item.quantidadeVendas, 0);

    return Object.entries(acumuladoPorTipo).map(([tipo, dados]) => ({
      tipoImovel: tipo,
      totalValor: Math.round(dados.totalValor * 100) / 100,
      percentualPorValor: Math.round((dados.totalValor / valorGeral) * 100 * 100) /
100,
      quantidadeVendas: dados.quantidadeVendas,
```

```
        percentualPorQuantidade: Math.round((dados.quantidadeVendas / quantidadeGeral) *
100 * 100) / 100,
    }));
  }
}
```

```
module.exports = TipoImovelController;
```

src/routes/vendasPorTipo.js

```
const express = require('express');
const TipoImovelController = require('../controllers/TipoImovelController');
```

```
const router = express.Router();
```

```
router.get('/', async (req, res) => {
  try {
    const resultado = await TipoImovelController.percentualPorTipo();
    res.status(200).json(resultado);
  } catch (erro) {
    res.status(500).json({ erro: erro.message });
  }
});
```

```
module.exports = router;
```

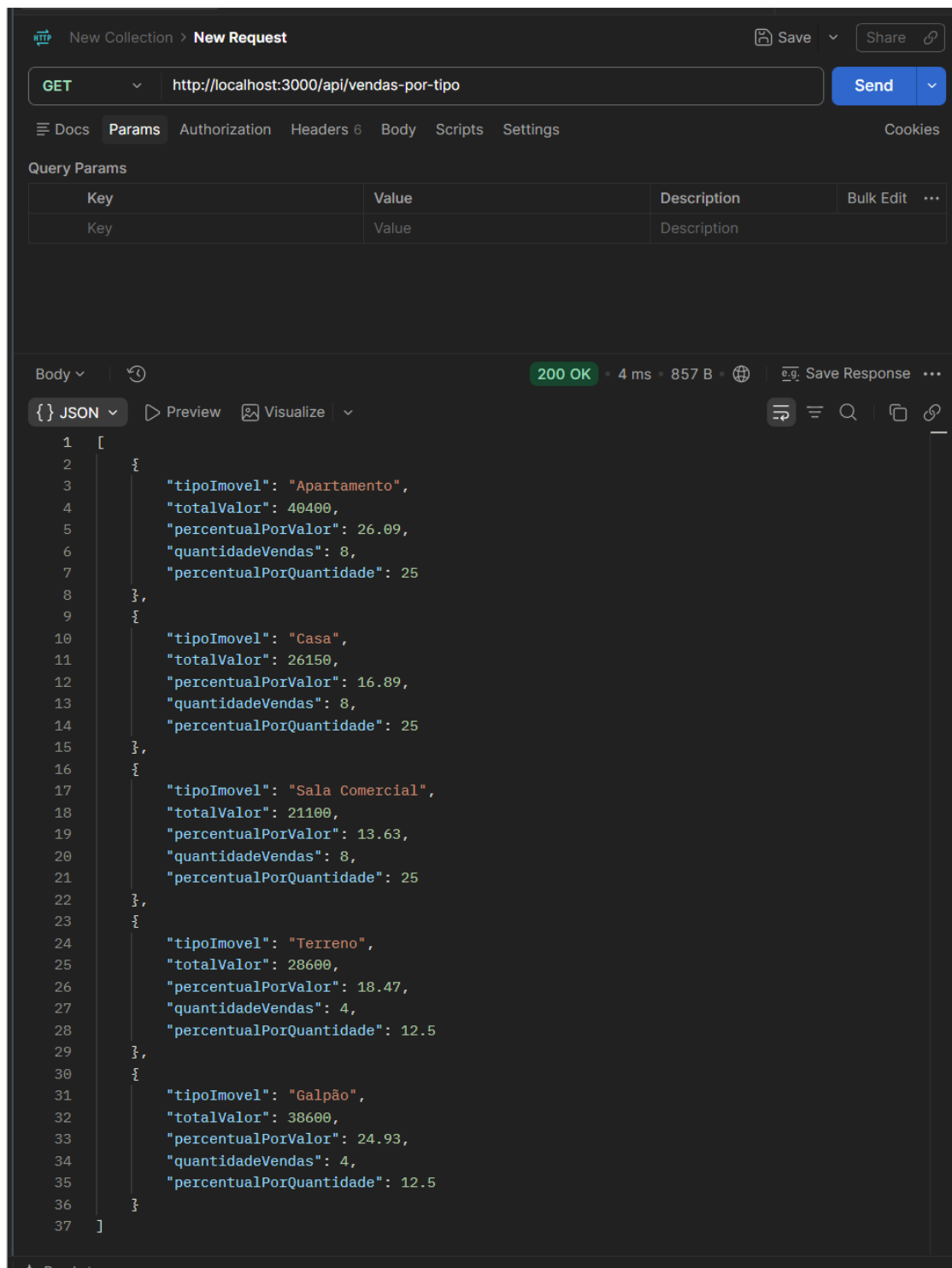


Figura 4 — Teste do serviço C: GET /api/vendas-por-tipo (200 OK)

10. Retorno em JSON via REST/GET e Testes (item d)

Os três serviços respondem a requisições GET e devolvem JSON.

Cada rota chama o método do seu controller e responde com `res.status(200).json(...)`. Em caso de falha, por exemplo com o banco de dados indisponível, a rota responde com o status 500 e um objeto `{ "erro": "...", ... }`, em vez de derrubar o servidor. Os testes foram realizados no Postman, com o servidor em execução (`npm start`) e o banco carregado pelos scripts `schema.sql` e `seed.sql` (32 pagamentos). As Figuras 2, 3 e 4 mostram, em cada chamada, a URL, o status 200 OK e o JSON retornado.

Como conferência de consistência entre os serviços, a soma dos valores retornados é a mesma nos três, R\$ 154.850,00, igual ao total dos 32 pagamentos cadastrados:

Serviço	Rota (GET)	Gráfico	Itens retornados	Soma dos valores
A — item (a)	<code>/api/soma-por-imovel</code>	Barras	8 imóveis	R\$ 154.850,00
B — item (b)	<code>/api/vendas-por-mes</code>	Linhas	6 meses	R\$ 154.850,00
C — item (c)	<code>/api/vendas-por-tipo</code>	Pizza	5 tipos de imóvel	R\$ 154.850,00

Tabela 1 — Resumo dos testes dos três serviços

No serviço C, os percentuais por quantidade somam exatamente 100%, e os percentuais por valor somam 100,01%, diferença que decorre apenas do arredondamento de cada tipo para duas casas decimais.

11. Ajustes em Relação à Etapa 1

Alterações feitas durante a Etapa 2 sobre os artefatos entregues na Parte 1.

- **Codificação de caracteres.** O banco passou a ser criado com `CHARACTER SET utf8mb4` e o pool de conexão com `charset: 'utf8mb4'`, para que nomes acentuados, como “Galpão”, sejam gravados e retornados corretamente no JSON.
- **Especificação OpenAPI.** Os exemplos de resposta foram atualizados para refletir o retorno real dos serviços implementados: o serviço B passou a devolver os campos `mesAno` e `total`, o serviço C passou a devolver as duas leituras de percentual e o total do imóvel 1 no exemplo do serviço A foi corrigido para 20.600. A versão final é apresentada a seguir.
- **Documentação dos models.** O guia das classes de model, citado na seção 4 como `src/models/README.md`, foi movido para `docs/modelos.md`.

docs/openapi.yaml — versão final

openapi: 3.0.3

info:

title: API de Transações Imobiliárias
description: Serviços REST para consulta de transações imobiliárias.
version: 1.0.0

servers:

- url: http://localhost:3000

paths:

/api/soma-por-imovel:

get:

summary: Soma das vendas por imóvel

responses:

'200':

description: Lista com o total de vendas por imóvel.

content:

application/json:

example:

- codigo_imovel: 1
descricao_imovel: "Apartamento 100m2 em condomínio fechado"
total_vendas: 20600
- codigo_imovel: 2
descricao_imovel: "Apartamento 60m2 no centro"
total_vendas: 19800
- codigo_imovel: 3
descricao_imovel: "Casa 3 quartos com quintal"
total_vendas: 13400
- codigo_imovel: 4
descricao_imovel: "Casa geminada 2 quartos"
total_vendas: 12750
- codigo_imovel: 5
descricao_imovel: "Sala comercial 40m2 no centro"
total_vendas: 10350
- codigo_imovel: 6
descricao_imovel: "Sala comercial em galeria"
total_vendas: 10750
- codigo_imovel: 7
descricao_imovel: "Terreno 500m2 em bairro residencial"
total_vendas: 28600
- codigo_imovel: 8
descricao_imovel: "Galpão industrial 800m2"
total_vendas: 38600

/api/vendas-por-mes:

get:

summary: Total de vendas por mês e ano

responses:

'200':

description: Lista com o total de vendas agrupadas por mês e ano.

content:

application/json:

example:

- mesAno: "08/2023"
total: 16100
- mesAno: "09/2023"
total: 26700
- mesAno: "10/2023"
total: 23650
- mesAno: "11/2023"
total: 29000
- mesAno: "12/2023"

```
total: 33500
- mesAno: "01/2024"
total: 25900
```

/api/vendas-por-tipo:

get:

summary: Percentual de vendas por tipo de imóvel

responses:

'200':

description: >-

Lista, para cada tipo de imóvel, o valor total vendido e a quantidade de vendas, com o percentual de cada um sobre o total geral. O percentual é calculado pelas

duas leituras do enunciado: sobre o valor total das vendas (percentualPorValor)

e sobre a quantidade de vendas (percentualPorQuantidade).

content:

application/json:

example:

```
- tipoImovel: "Apartamento"
  totalValor: 40400
  percentualPorValor: 26.09
  quantidadeVendas: 8
  percentualPorQuantidade: 25
- tipoImovel: "Casa"
  totalValor: 26150
  percentualPorValor: 16.89
  quantidadeVendas: 8
  percentualPorQuantidade: 25
- tipoImovel: "Sala Comercial"
  totalValor: 21100
  percentualPorValor: 13.63
  quantidadeVendas: 8
  percentualPorQuantidade: 25
- tipoImovel: "Terreno"
  totalValor: 28600
  percentualPorValor: 18.47
  quantidadeVendas: 4
  percentualPorQuantidade: 12.5
- tipoImovel: "Galpão"
  totalValor: 38600
  percentualPorValor: 24.93
  quantidadeVendas: 4
  percentualPorQuantidade: 12.5
```

12. Vídeo de Apresentação

Vídeo publicado no YouTube, com duração de 2 min 48 s (limite: 3 minutos).

<https://youtu.be/4Ovx Ct7qC5c>

O vídeo apresenta cada uma das três requisições e o respectivo resultado em JSON, o código-fonte e a estrutura do banco de dados, conforme solicitado no enunciado.

13. Repositório do Projeto

Código-fonte completo, scripts SQL, especificação OpenAPI, diagrama UML e prints dos testes.

<https://github.com/FelipeCJBoB/how7-backend-imobiliario>

Para executar o projeto: rodar `db/schema.sql` e `db/seed.sql` no MySQL, copiar `.env.example` para `.env` com as credenciais do banco, instalar as dependências com `npm install` e iniciar o servidor com `npm start`. Os serviços ficam disponíveis em `http://localhost:3000`.